

Technical Documentation of FPGA-Based Autonomous Intersection Controller

Group A7 – Md Azadul Islam | Faysal Ahammed Tonmoy | MD Jubair Salehin Razin | MD Rafiul Islam
Hochschule Hamm-Lippstadt
Electronic Engineering
Hardware Engineering Lab

Abstract—This technical documentation presents the design and implementation of an FPGA-based Roadside Unit (RSU) intersection reservation controller for autonomous vehicles. The system represents a four-way intersection with North, South, East, and West vehicle requests. The North-South direction is considered the main road, while the East-West direction is considered the side road.

The RSU controller grants access to the intersection critical zone using deterministic timing, main-road priority, side-road fairness, and conflict-free safety logic. The controller is implemented in VHDL, verified using a VHDL testbench in QuestaSim, and prepared for FPGA prototype demonstration using switches, buttons, LEDs, and optional debug displays.

In addition, the documentation describes the schematic and PCB layout concept for a future FPGA-based evaluation board.

Index Terms—FPGA, VHDL, RSU, Roadside Unit, autonomous vehicles, intersection reservation, finite state machine, traffic control, PCB design, QuestaSim.

I. INTRODUCTION

Autonomous vehicles require safe and deterministic coordination when passing through road intersections. In conventional traffic systems, human drivers follow traffic lights and road signs. However, in a future autonomous traffic environment, permission to enter an intersection can be handled directly by a Roadside Unit (RSU).

The RSU can communicate with vehicles and provide access permission based on request signals, priority rules, timing constraints, and safety requirements.

The main idea of this project is to design a hardware-level RSU controller for a four-way intersection. The controller receives vehicle request signals from the North, South, East, and West directions and generates grant signals that allow vehicles to enter the intersection.

The center of the intersection is modeled as a critical zone. This critical zone must never be accessed by conflicting road groups at the same time.

The project focuses on an FPGA-based implementation because FPGAs provide deterministic hardware behavior, parallel signal processing, and predictable timing. The controller is described using VHDL and verified using simulation.

The design is also prepared for practical FPGA demonstration using switches or buttons as request inputs and LEDs as grant outputs.

II. MOTIVATION

Autonomous intersection control is an important topic because intersections are safety-critical areas in road traffic. Vehicles arriving from different directions may create conflicts if access is not coordinated correctly.

A conventional traffic light system gives permission to road users using visual signals. In an autonomous vehicle scenario, permission can be given directly to vehicles by an RSU controller.

The motivation of this project is to implement a simplified but deterministic RSU controller using digital hardware. The controller must satisfy three important goals:

- **Safety:** Conflicting road groups must not receive grants at the same time.
- **Priority:** The main road should be served first when both road groups request access.
- **Fairness:** The side road must not wait indefinitely.

The project demonstrates how a finite state machine and timing counters can be used to realize a reservation-based intersection controller on FPGA hardware.

III. SYSTEM SCENARIO

The system represents a four-way intersection with four incoming directions: North, South, East, and West. The North-South road is defined as the main road, and the East-West road is defined as the side road.

A vehicle must request permission from the RSU before entering the center of the intersection.

The center of the intersection is modeled as a square-shaped critical zone. The RSU controller is responsible for ensuring that conflicting road groups do not enter this critical zone at the same time.

IV. PROJECT OBJECTIVES

The main objective of the project is to design and validate an FPGA-based RSU controller for autonomous intersection reservation.

The detailed objectives are listed below:

- Design a VHDL-based RSU controller.
- Receive vehicle request signals from four directions.
- Group North and South as the main road.
- Group East and West as the side road.
- Give priority to the main road when both groups request access.

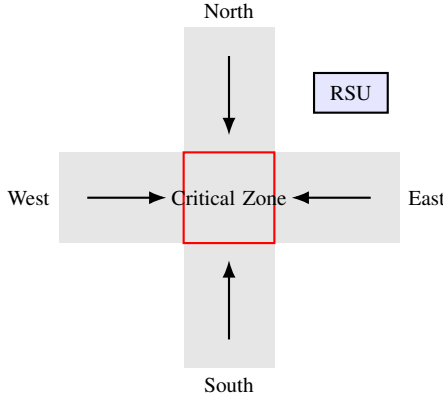


Fig. 1. Four-way intersection scenario with critical zone and RSU controller.

TABLE I
FUNCTIONAL REQUIREMENTS

ID	Requirement
FR1	The RSU shall receive vehicle request signals from North, South, East, and West.
FR2	The RSU shall group North and South as the main road.
FR3	The RSU shall group East and West as the side road.
FR4	The RSU shall grant the main road first when both main and side roads request access.
FR5	The RSU shall grant the side road immediately when the main road has no waiting vehicle.
FR6	The RSU shall ensure that the side road does not wait more than 20 seconds.
FR7	The RSU shall not allow main-road and side-road grants at the same time.
FR8	The RSU shall use a 5-second crossing time before allowing a conflicting road group.
FR9	The RSU shall serve the main road for a maximum of 20 seconds when the side road is waiting.
FR10	The RSU shall serve the side road for a maximum of 10 seconds before returning to the main road if main-road vehicles are waiting.

- Serve the side road when no main-road vehicle is waiting.
- Ensure that the side road does not wait longer than the maximum allowed time.
- Prevent simultaneous grants to conflicting road groups.
- Verify the design using VHDL testbench simulation.
- Prepare the design for FPGA prototype testing.
- Develop a schematic and PCB layout concept for future hardware implementation.

V. SYSTEM REQUIREMENTS

A. Functional Requirements

The functional requirements define the expected behavior of the RSU controller.

B. Non-Functional Requirements

The non-functional requirements describe implementation and quality constraints.

TABLE II
NON-FUNCTIONAL REQUIREMENTS

ID	Requirement
NFR1	The design shall be implemented in VHDL.
NFR2	The design shall be suitable for FPGA implementation.
NFR3	The design shall be readable and modular.
NFR4	The design shall be testable using a VHDL testbench.
NFR5	The design shall be observable on FPGA using LEDs or a seven-segment display.
NFR6	The design shall be simple enough to be transferred later to a custom PCB or evaluation-board concept.

TABLE III
INPUT SIGNALS

Signal	Type	Description
clk	Input	System clock signal.
reset	Input	Resets the controller to its initial state.
N_req	Input	Request from the North vehicle.
S_req	Input	Request from the South vehicle.
E_req	Input	Request from the East vehicle.
W_req	Input	Request from the West vehicle.

TABLE IV
OUTPUT SIGNALS

Signal	Type	Description
N_grant	Output	Grant signal for the North vehicle.
S_grant	Output	Grant signal for the South vehicle.
E_grant	Output	Grant signal for the East vehicle.
W_grant	Output	Grant signal for the West vehicle.

VI. SYSTEM INPUTS AND OUTPUTS

The RSU controller receives request inputs from four vehicle directions and produces grant outputs for the corresponding directions.

A. Input Signals

The grouped request equations are:

$$\text{Main_req} = \text{N_req} \vee \text{S_req} \quad (1)$$

$$\text{Side_req} = \text{E_req} \vee \text{W_req} \quad (2)$$

B. Output Signals

The grouped grant equations are:

$$\text{Main_grant} = \text{N_grant} \vee \text{S_grant} \quad (3)$$

$$\text{Side_grant} = \text{E_grant} \vee \text{W_grant} \quad (4)$$

VII. TIMING REQUIREMENTS

The controller uses timing parameters to provide deterministic service and clearing behavior.

A vehicle requires 5 seconds to clear the intersection critical zone. Therefore, when the side road is waiting, the controller

TABLE V
TIMING PARAMETERS

Parameter	Value	Meaning
T_{cross}	5 s	Time required for a vehicle to clear the intersection.
T_{main}	20 s	Maximum main-road service time.
T_{side}	10 s	Maximum side-road service time.
$W_{\text{side,max}}$	20 s	Maximum allowed side-road waiting time.

must stop granting new main-road vehicles early enough so that the side road can receive access before the maximum waiting time expires.

The latest possible main-road grant is calculated as:

$$t_{\text{latest main grant}} = W_{\text{side,max}} - T_{\text{cross}} \quad (5)$$

$$t_{\text{latest main grant}} = 20 \text{ s} - 5 \text{ s} = 15 \text{ s} \quad (6)$$

This means that if the side road has already waited 15 seconds, the RSU must stop granting new main-road vehicles and move to the clearing state.

VIII. SAFETY AND FAIRNESS RULES

A. Safety Rule

The most important safety requirement is that the main road and side road must never receive grants at the same time. This rule prevents conflicting vehicles from entering the intersection critical zone simultaneously.

$$(N_{\text{grant}} \vee S_{\text{grant}}) \wedge (E_{\text{grant}} \vee W_{\text{grant}}) = 0 \quad (7)$$

This can also be written as:

$$\text{Main_grant} \wedge \text{Side_grant} = 0 \quad (8)$$

B. Fairness Rule

Although the main road has priority, the side road must not wait forever. The fairness requirement is:

$$\text{side_wait_timer} \leq 20 \text{ s} \quad (9)$$

The side-road waiting timer is used to detect how long the side road has been requesting access. If the waiting time approaches the maximum limit, the controller stops giving new main-road grants and prepares to serve the side road.

IX. CONTROLLER ARCHITECTURE

The RSU controller consists of several internal blocks. The request grouping logic combines individual vehicle requests into main-road and side-road request groups.

The finite state machine decides which road group receives access. Timer units control service duration, crossing time, and side-road waiting time. Finally, the grant output logic activates the correct grant signals.

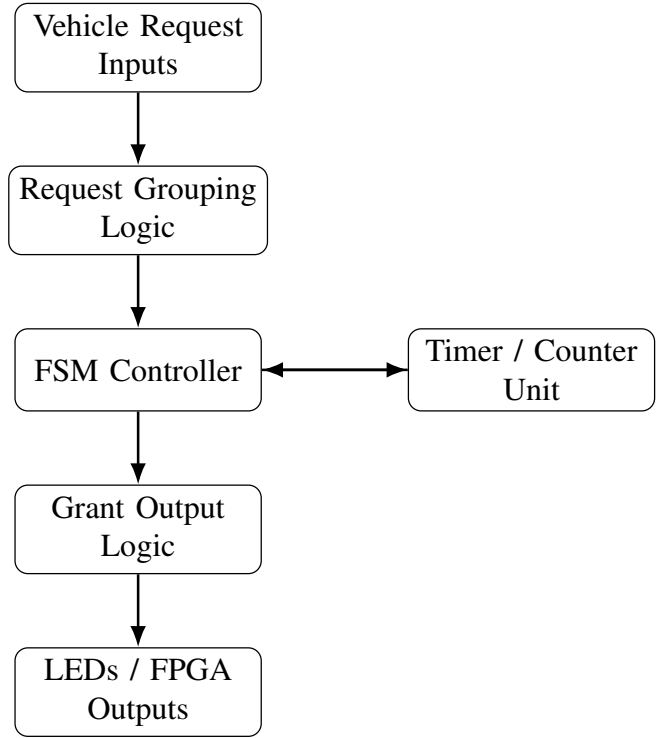


Fig. 2. Internal architecture of the FPGA-based RSU controller.

TABLE VI
FSM STATE DESCRIPTION

State	Description
IDLE	No active request is present.
MAIN_SERVICE	The controller grants main-road vehicles.
MAIN_CLEAR	The controller stops new main-road grants and waits for the intersection to clear.
SIDE_SERVICE	The controller grants side-road vehicles.
SIDE_CLEAR	The controller stops new side-road grants and waits for the intersection to clear.

X. FINITE STATE MACHINE DESIGN

The RSU controller is implemented using a finite state machine. The FSM contains five states:

- IDLE
- MAIN_SERVICE
- MAIN_CLEAR
- SIDE_SERVICE
- SIDE_CLEAR

A. State Transition Explanation

When the controller is in IDLE, it checks the request inputs. If a main-road request exists, the controller enters MAIN_SERVICE.

If only a side-road request exists, the controller enters SIDE_SERVICE.

During MAIN_SERVICE, the main road receives grant signals. If the side road is waiting and the side waiting timer reaches the defined threshold, the controller enters MAIN_CLEAR.

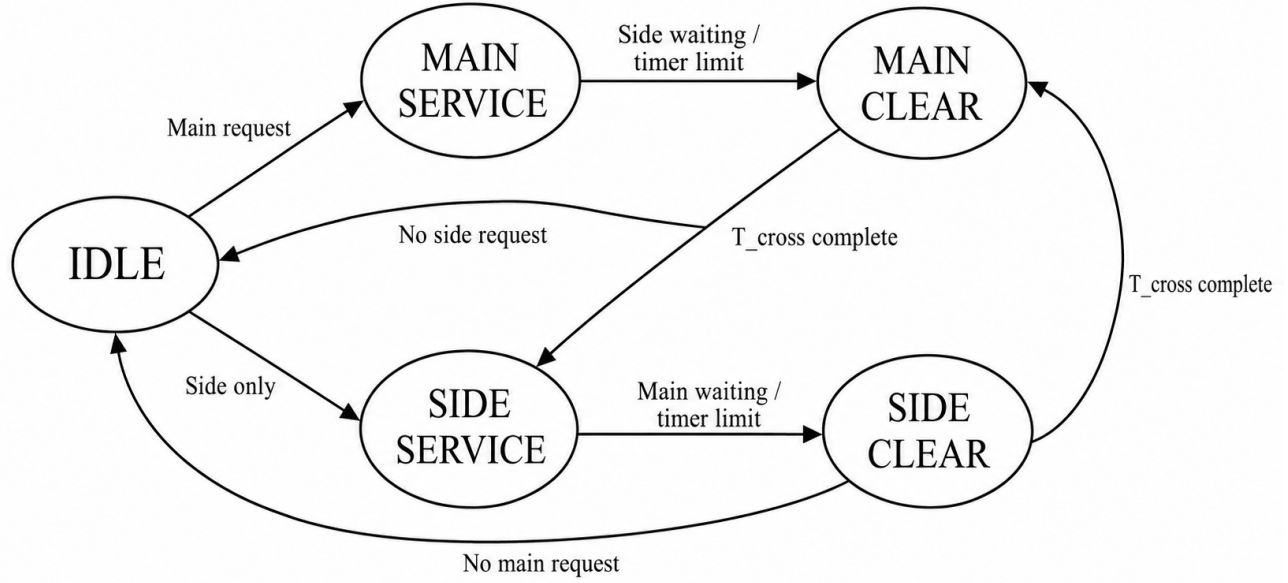


Fig. 3. FSM state diagram of the RSU controller.

After the crossing time is complete, the controller enters SIDE_SERVICE. The same principle is used when returning from side-road service to main-road service.

XI. VHDL IMPLEMENTATION

The RSU controller is implemented in VHDL. The design is divided into input grouping logic, FSM logic, timer logic, and output grant logic.

A. Entity Declaration

The main VHDL entity contains the clock, reset, request inputs, and grant outputs.

```

1  entity rsu_controller is
2  port (
3      clk      : in  std_logic;
4      reset    : in  std_logic;
5
6      N_req    : in  std_logic;
7      S_req    : in  std_logic;
8      E_req    : in  std_logic;
9      W_req    : in  std_logic;
10
11     N_grant   : out std_logic;
12     S_grant   : out std_logic;
13     E_grant   : out std_logic;
14     W_grant   : out std_logic;
15 );
16
17 end rsu_controller;

```

Listing 1. Simplified VHDL entity of the RSU controller.

B. Internal Signals

The design uses internal signals for request grouping, FSM states, and timing counters.

```

1  type state_type is (
2      IDLE,
3      MAIN_SERVICE,
4      MAIN_CLEAR,
5      SIDE_SERVICE,
6      SIDE_CLEAR
7  );
8
9  signal current_state : state_type;
10 signal next_state    : state_type;
11
12 signal main_req : std_logic;
13 signal side_req : std_logic;
14
15 signal main_timer      : integer
16     range 0 to 20;
17 signal side_timer      : integer
18     range 0 to 10;
19 signal side_wait_timer : integer
20     range 0 to 20;
21 signal clear_timer     : integer
22     range 0 to 5;

```

Listing 2. Example internal signal structure.

C. Request Grouping Logic

The request grouping logic combines the individual direction requests into main-road and side-road requests.

```

1  main_req <= N_req or S_req;
2  side_req <= E_req or W_req;

```

Listing 3. Request grouping logic.

D. Timer Concept

The FPGA board usually runs with a high-frequency clock. Therefore, a clock divider is required to create a visible one-second tick for testing.

TABLE VII
FPGA PROTOTYPE INPUT/OUTPUT MAPPING

FPGA Element	Signal	Purpose
Switch/Button 1	N_req	North vehicle request.
Switch/Button 2	S_req	South vehicle request.
Switch/Button 3	E_req	East vehicle request.
Switch/Button 4	W_req	West vehicle request.
LED 1	N_grant	North grant output.
LED 2	S_grant	South grant output.
LED 3	E_grant	East grant output.
LED 4	W_grant	West grant output.
Reset button	reset	Controller reset.
Seven-segment display	Debug output	Optional state or timer display.

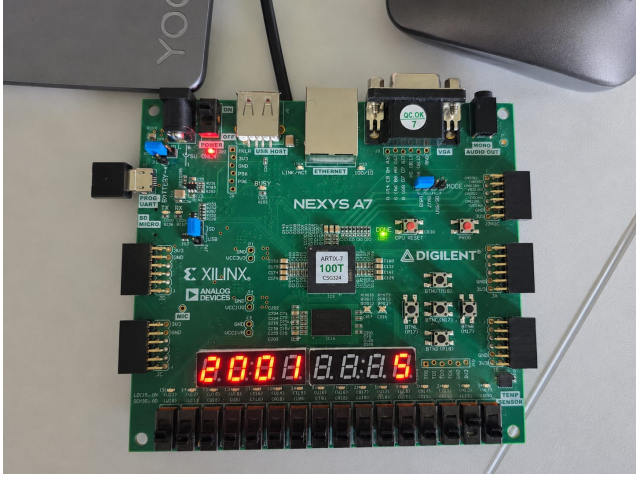


Fig. 4. FPGA prototype setup with switches, buttons, and LEDs.

For a clock frequency of f_{clk} , the number of clock cycles required to generate a one-second tick is:

$$N_{1s} = \frac{f_{\text{clk}}}{1 \text{ Hz}} \quad (10)$$

For example, if the FPGA clock is 100 MHz, one second corresponds to:

$$1 \text{ s} = 100,000,000 \text{ clock cycles} \quad (11)$$

E. Output Grant Logic

The output grant logic activates only the allowed grant signals according to the current state and active requests.

During MAIN_SERVICE, only the North and South grant signals may be active. During SIDE_SERVICE, only the East and West grant signals may be active.

XII. FPGA PROTOTYPE IMPLEMENTATION

The FPGA prototype uses switches or buttons to represent vehicle request inputs. LEDs are used to display grant outputs. A reset button is used to restart the controller.

Optionally, a seven-segment display can show the current FSM state or timer value.

TABLE VIII
SIMULATION TEST CASES

TC	Input Condition	Expected Result
TC1	No request	No grant is active.
TC2	Only North request	North grant is active.
TC3	Only South request	South grant is active.
TC4	Only East request	East grant is active.
TC5	Only West request	West grant is active.
TC6	North and South requests	Main-road grants are active.
TC7	East and West requests	Side-road grants are active if the main road is empty.
TC8	Main-road and side-road requests occur together	The main road is served first.
TC9	The side road waits for 15 seconds	New main-road grants stop.
TC10	The side road reaches the maximum waiting time	The side road receives a grant.
TC11	Main-road and side-road grants are monitored	They are never active together.

XIII. SIMULATION AND VERIFICATION

The design is verified using a VHDL testbench in QuestaSim. The testbench applies different request combinations and observes the grant outputs.

The main purpose of verification is to confirm that the controller satisfies the safety, priority, timing, and fairness requirements.

A. Testbench Concept

The testbench performs the following tasks:

- Generates the system clock.
- Applies reset at the beginning of the simulation.
- Applies different vehicle request combinations.
- Observes the grant outputs.
- Checks that main-road and side-road grants are never active together.
- Checks that the side road receives service before the maximum waiting time.

B. Verification Test Cases

C. Safety Verification

The safety property is verified by ensuring that the following unsafe condition never becomes true:

$$\text{Main_grant} \wedge \text{Side_grant} = 1 \quad (12)$$

Therefore, the required safety result is:

$$\text{Main_grant} \wedge \text{Side_grant} = 0 \quad (13)$$

D. Fairness Verification

The fairness property is verified by checking that the side-road request does not wait longer than the allowed maximum waiting time:

$$\text{side_wait_timer} \leq W_{\text{side,max}} \quad (14)$$

XIV. SCHEMATIC DESIGN

The schematic design represents the future FPGA evaluation board or custom PCB concept for the RSU controller.

The schematic includes the required hardware components for implementing and observing the controller.

A. Main Schematic Blocks

The schematic contains the following main blocks:

- FPGA device.
- Clock source.
- Reset circuit.
- Power supply circuit.
- Programming interface.
- Vehicle request input switches or buttons.
- Grant output LEDs.
- Optional seven-segment display.
- Decoupling capacitors and supporting passive components.

B. Power Supply Sheet

This sheet contains the input protection, voltage regulators, decoupling capacitors, and power test points. These components provide stable supply voltages for the FPGA and the other board circuits.

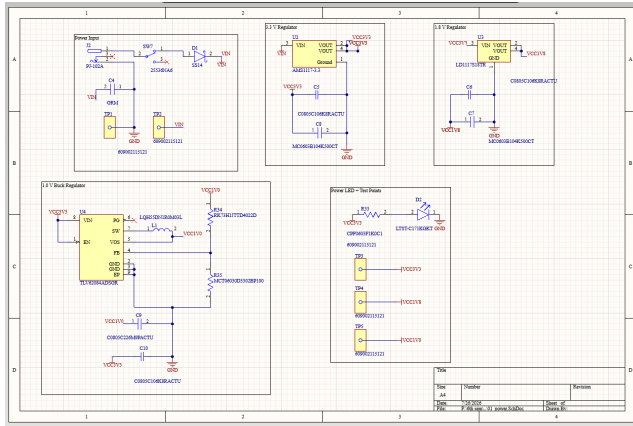


Fig. 5. Power supply and voltage regulation schematic sheet.

C. FPGA Core Sheet

This sheet contains the FPGA device, clock oscillator, JTAG connector, and required power connections. The FPGA performs the RSU control logic, while JTAG is used for programming and debugging.

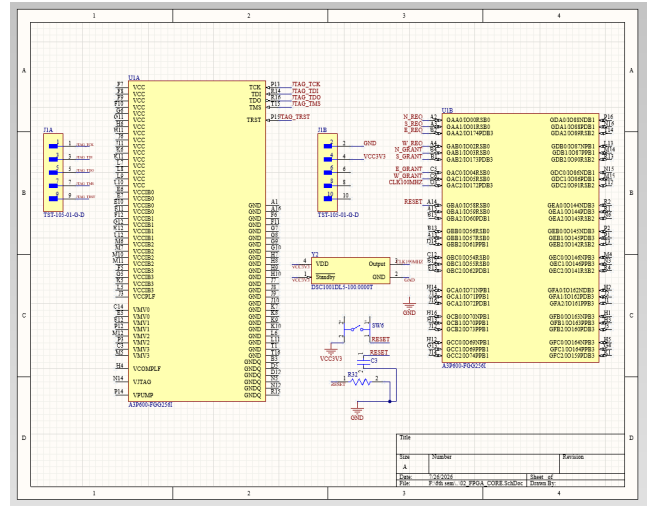


Fig. 6. FPGA core, clock, and JTAG programming schematic sheet.

D. Request and Grant I/O Sheet

This sheet contains the North, South, East, and West request inputs and grant outputs. The switches provide vehicle requests, while the output connections control the direction grant indicators.

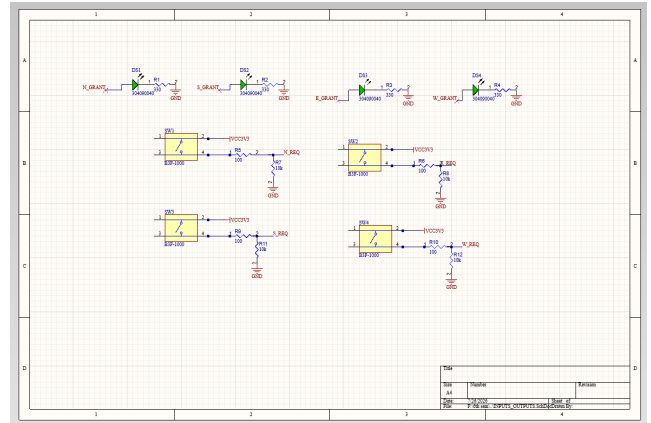


Fig. 7. Vehicle request input and grant output schematic sheet.

E. Debug Display Sheet

This sheet contains the seven-segment displays and their supporting connections. The displays can be used to show the current FSM state or timer values during testing.

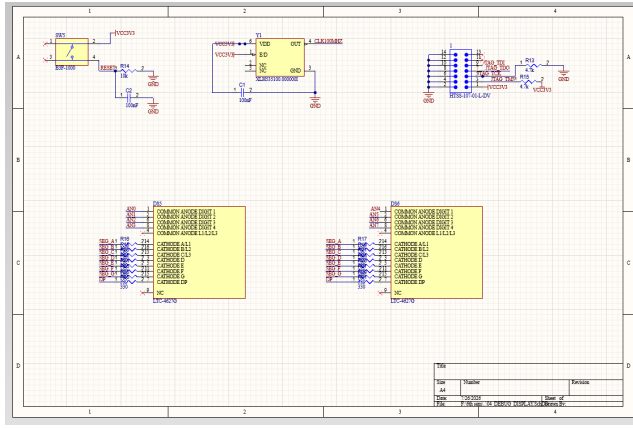


Fig. 8. Seven-segment debug display schematic sheet.

F. Top-Level Schematic Sheet

This sheet connects the power supply, FPGA core, request and grant I/O, and debug display blocks. It shows the complete hierarchical connection of the RSU controller board.

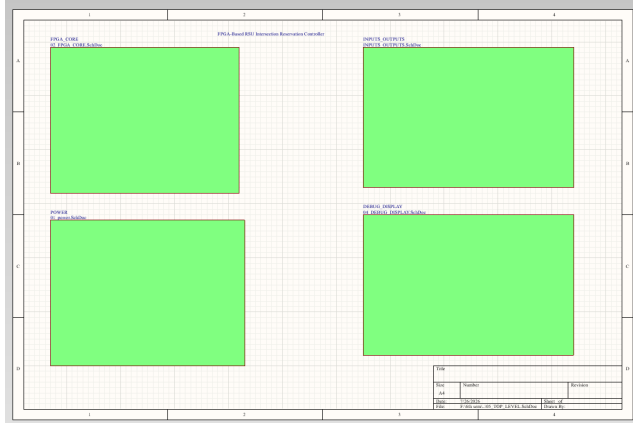


Fig. 9. Top-level schematic of the complete RSU controller board.

XV. PCB LAYOUT DESIGN

The PCB layout is designed to support the FPGA-based RSU controller demonstration. The FPGA is placed as the central control component.

Input switches or buttons represent vehicle requests, and output LEDs represent grant signals. The board also includes power, clock, reset, and programming connections.

A. Component Placement

The FPGA should be placed near the center of the PCB. Power regulation components should be placed close to the power input and FPGA power pins.

Switches and LEDs should be placed near the board edges so that they are easy to access and observe during testing.

B. Routing Considerations

Power traces should be wide enough to carry the required current. Signal traces for switches and LEDs can be routed as normal digital I/O lines.

Clock routing should be kept short and clean to reduce noise and timing problems.

C. Design Rule Check

After routing, design rule checks should be performed to detect clearance errors, unconnected nets, short circuits, and footprint-related problems.

Only a 3D rendered export of the assembled PCB was available for this documentation. Separate top-layer and bottom-layer copper routing screenshots were not exported.

Figure 10 presents the QuestaSim simulation waveform of the FPGA-based RSU intersection controller.

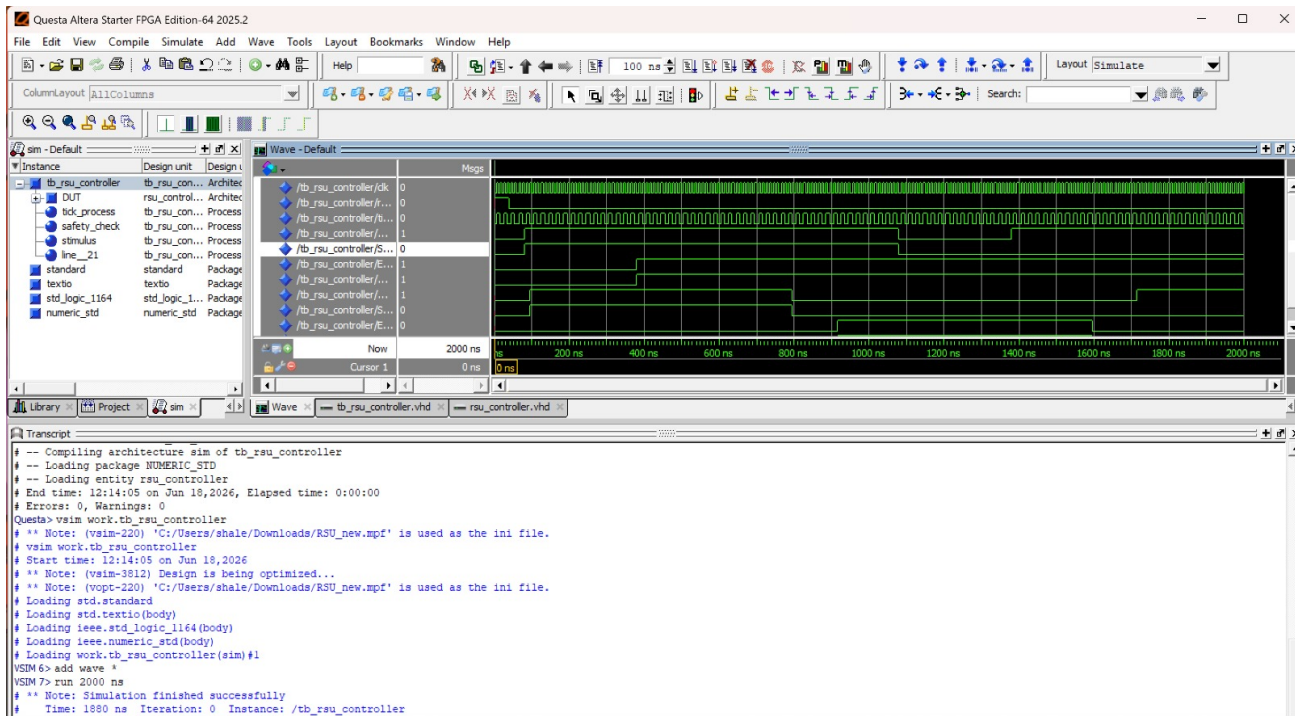


Fig. 10. QuestaSim simulation waveform of the RSU controller.

APPENDIX B: PCB LAYOUT

Figure 11 presents the complete 3D rendered PCB layout of the FPGA-based RSU controller.

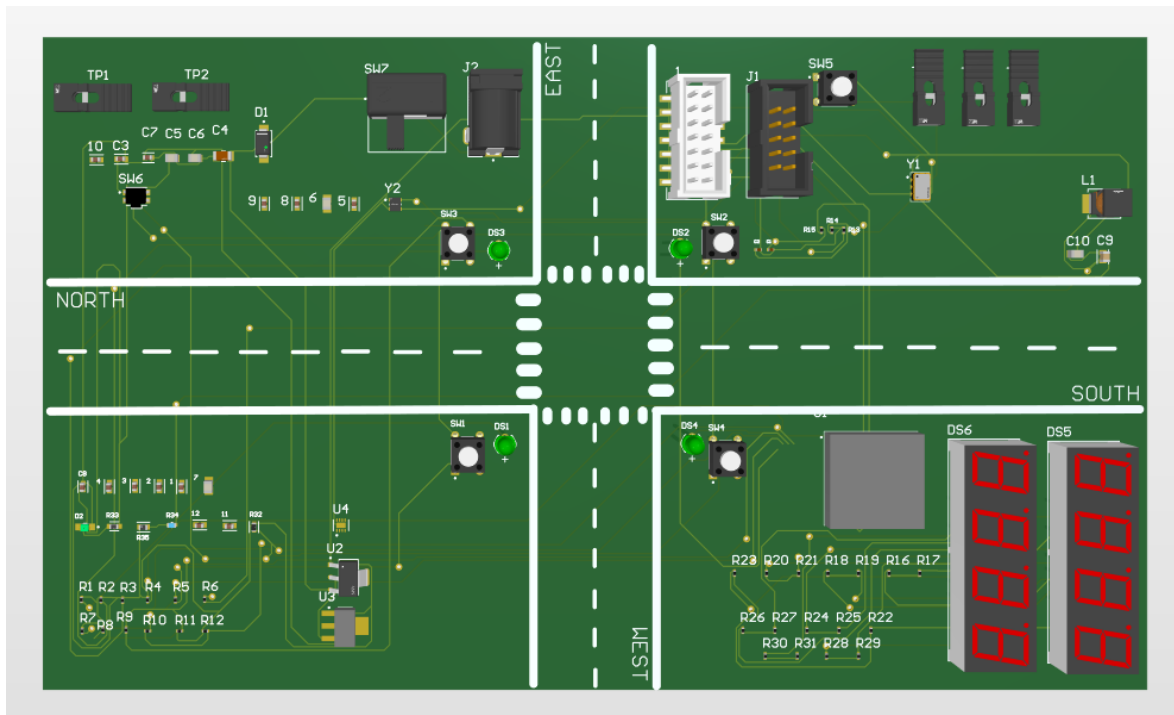


Fig. 11. Complete 3D PCB layout of the FPGA-based RSU controller.

TABLE IX
REQUIREMENT VALIDATION SUMMARY

Requirement	Status	Evidence
Main-road priority	Passed	Simulation and FPGA prototype test.
Side-road fairness	Passed	Side-road waiting timer test.
No conflicting grants	Passed	Safety equation and simulation test.
Crossing time applied	Passed	FSM clearing states and timer test.
FPGA implementation	Passed	Physical FPGA prototype test.
PCB concept	Completed	Schematic and PCB layout design.

XVI. RESULTS

The results section summarizes the outcomes of the VHDL simulation, FPGA prototype testing, and PCB design work.

A. Simulation Results

The simulation verified that the RSU controller follows the required state transitions and timing behavior.

The simulation showed that the controller grants the main road first when both main-road and side-road requests are active. The side road receives access when the main road is empty or when the side-road waiting timer reaches the fairness threshold.

The simulation also confirmed that main-road and side-road grant signals are never active at the same time.

B. FPGA Prototype Results

The FPGA prototype demonstrated the controller behavior using physical input switches or buttons and LED outputs. The request inputs were applied manually, and the corresponding grant outputs were observed using LEDs.

The prototype demonstrated the following behavior:

- North and South requests activated the main-road outputs.
- East and West requests activated the side-road outputs.
- Main-road requests received priority during simultaneous requests.
- The reset input returned the controller to the IDLE state.
- Conflicting grant outputs were not activated simultaneously.

C. PCB Design Results

The PCB concept includes the required hardware blocks for a future standalone RSU controller board.

The schematic and PCB design contain the FPGA, power supply, oscillator, programming interface, request switches, grant LEDs, reset circuit, and optional debug display.

The schematic and 3D PCB view demonstrate that the design can be transferred from simulation and FPGA-board testing to a custom hardware implementation.

D. Requirement Validation

XVII. DISCUSSION

The implemented RSU controller demonstrates how an autonomous intersection reservation concept can be realized using deterministic digital hardware.

The FSM-based architecture makes the design simple, readable, modular, and testable. The main safety rule ensures that conflicting road groups are never granted access at the same time.

The clearing states provide sufficient time for a vehicle to leave the intersection before a conflicting road group receives permission.

The fairness mechanism prevents the side road from waiting indefinitely, even though the main road has higher priority. When the side-road waiting time approaches the defined limit, the controller stops issuing new main-road grants and prepares to serve the side road.

The FPGA implementation is suitable for this type of controller because its timing behavior is predictable and the control logic runs directly in hardware.

Compared with a software-only implementation, the FPGA design can provide deterministic response behavior, parallel signal processing, and direct signal-level control.

However, the implemented system is a simplified prototype. It assumes that request signals are already available and does not implement real wireless communication between vehicles and the RSU.

It also does not include vehicle-speed estimation, turning-direction management, pedestrian requests, or emergency-vehicle priority.

The timing values used in the controller are simplified values for simulation and demonstration. A real traffic intersection would require timing values based on vehicle speed, intersection dimensions, communication delay, braking distance, and additional safety margins.

XVIII. CONCLUSION AND FUTURE WORK

This project successfully designed and implemented an FPGA-based Roadside Unit intersection reservation controller for autonomous vehicles.

The controller receives request signals from the North, South, East, and West directions and generates grant signals according to priority, fairness, timing, and safety requirements.

The North-South road is treated as the main road, while the East-West road is treated as the side road. The controller gives priority to the main road when requests from both road groups are active.

At the same time, the fairness mechanism ensures that the side road is not blocked indefinitely.

The controller prevents simultaneous conflicting grants and uses clearing states to ensure that the intersection critical zone is empty before access is transferred to another road group.

The VHDL implementation was verified using a testbench in QuestaSim. The design was also prepared for FPGA prototype testing using switches, buttons, LEDs, and optional debug displays.

In addition, a schematic and PCB layout concept was developed for a possible future standalone hardware implementation.

Future development of the system can include:

- Adding separate left-turn, right-turn, and straight movement requests.
- Adding pedestrian crossing requests.
- Adding emergency-vehicle priority.
- Adding wireless Vehicle-to-Everything communication.
- Adding vehicle-speed and arrival-time estimation.
- Adding dynamic timing according to traffic density.
- Adding formal verification of safety properties.
- Implementing UART-based debugging.
- Implementing a seven-segment state and timer display.
- Manufacturing and testing the custom PCB.

REFERENCES

- [1] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with VHDL Design*. McGraw-Hill, 2009.
- [2] P. J. Ashenden, *The Designer's Guide to VHDL*. Morgan Kaufmann, 2008.
- [3] R. H. Katz and G. Borriello, *Contemporary Logic Design*. Pearson, 2005.
- [4] K. Dresner and P. Stone, "A multiagent approach to autonomous intersection management," *Journal of Artificial Intelligence Research*, vol. 31, pp. 591–656, 2008.
- [5] M. Lu, "Evaluation of intelligent road transport systems," *IET Intelligent Transport Systems*, vol. 5, no. 4, pp. 288–298, 2011.
- [6] C. Robertson, *Printed Circuit Board Designer's Reference: Basics*. Prentice Hall, 2003.